Cletus — Section 1, Day 5

- **Date:** [YYYY-MM-DD]
- **Course:** Week 1, Day 5 — Company brochure generator: JSON prompts, chaining LLM calls, streaming
- **Lab notebook:** week1/day5.ipynb + week1/week1 EXERCISE.ipynb (solution: week1/solutions/week1 SOLUTION.ipynb)
- **Time spent:** [e.g. 2h]

Key concepts

- **The business challenge** — given a company name and website, write a brochure for customers, investors and recruits.
- **Chaining LLM calls** — call 1 picks the relevant links, and call 2 writes the brochure from those pages. The output of one call feeds the next, which is the first step toward **agentic AI**.
- **Zero-, one- and multi-shot prompting** — give 0, 1 or several examples of the output you want inside the prompt. Day 5 uses **one-shot** (one example of the JSON).
- **JSON mode** — `response_format={"type": "json_object"}` makes the model reply with valid JSON, which you read with `json.loads()`. A stricter version, *Structured Outputs*, comes in Week 8.
- **LLMs for fuzzy judgement** — deciding which links are "relevant" would be hard to code by hand, but is easy for an LLM.
- **Truncating input** — cut long text (e.g. `[:5_000]`) to control tokens and cost.
- **Streaming** — `stream=True` sends the reply in chunks as it's written; read `chunk.choices[0].delta.content` to get the typewriter effect.
- **Tone is just the system prompt** — change it to "humorous, witty" and the same pipeline makes a funny brochure.

Code / examples

Pipeline: **get all links → LLM picks relevant links (JSON) → fetch those pages → LLM writes brochure → stream it**

Step 1, the LLM chooses links and replies in JSON:

```
import json
from openai import OpenAI
from scraper import fetch_website_links, fetch_website_contents

openai = OpenAI()

link_system_prompt = """You decide which links on a company website are relevant for a brochure
(About, Company, Careers pages). Respond in JSON like this example:
{"links": [{"type": "about page", "url": "https://full.url/about"}]}"""

def select_relevant_links(url):
    links = "\n".join(fetch_website_links(url))
    response = openai.chat.completions.create(
        model="gpt-5-nano",
        messages=[
            {"role": "system", "content": link_system_prompt},
            {"role": "user", "content": f"Links on {url} (some relative):\n{links}"},
        ],
```

```
        response_format={"type": "json_object"},
    )
    return json.loads(response.choices[0].message.content)
```

Step 2: landing page + each relevant page → one big user prompt → brochure, streamed:

```
stream = openai.chat.completions.create(model="gpt-4.1-mini", messages=messages, stream=True)
for chunk in stream:
    print(chunk.choices[0].delta.content or "", end="")
```

Exercise / challenge

Week 1 challenge: a technical tutor. Take a technical question, answer it with GPT (streamed) and with Llama 3.2 (local), then compare.

```
question = """Please explain what this code does and why:
yield from {book.get("author") for book in books if book.get("author")}"""

messages = [
    {"role": "system", "content": "You are a helpful technical tutor for Python, software
engineering, data science and LLMs."},
    {"role": "user", "content": question},
]

# GPT, streamed
for chunk in openai.chat.completions.create(model="gpt-4o-mini", messages=messages, stream=True):
    print(chunk.choices[0].delta.content or "", end="")

# Llama 3.2, local via Ollama
ollama = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
print(ollama.chat.completions.create(model="llama3.2",
messages=messages).choices[0].message.content)
```

Extra practice: - Make a brochure for another company. - Add a 3rd call that translates the brochure into French. - Use `input()` so the tutor asks you for the question.

Questions & blockers

Watch out for: - JSON mode needs the word **"JSON"** somewhere in your messages, or OpenAI returns an error. - Some links fail to scrape. Relative links like `/about` must become full URLs, and the prompt asks the LLM to do that. - Long pages use lots of tokens, so truncate the input.

To discuss with my partner

- Other content-generation uses at work: marketing copy, personalised emails, tutorials from a spec.
- What goes into our Week 1 `summary.md`?

Resources

- `week1/scraper.py` (`fetch_website_links`, `fetch_website_contents`)
- Prompting guide: https://platform.openai.com/docs/guides/prompt-engineering
- Other students' projects: `week1/community-contributions/`